Either & Railway-Oriented Programming

Either/Railway · Validate user id pipeline without exceptions · Build with Waqar

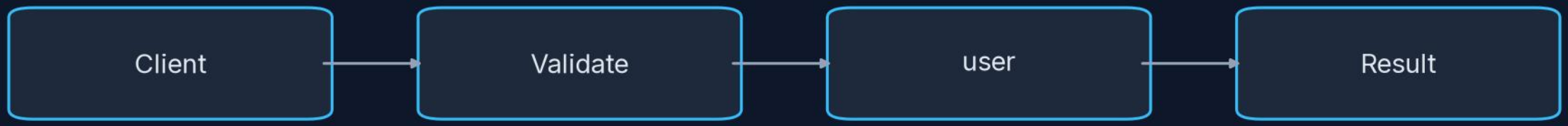
Today's Hook

- Problem: Validate user id pipeline without exceptions
- Pattern: Either/Railway — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Validate user id pipeline without exceptions
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Either/Railway: Validate user id pipeline without exceptions

Real-World Scenario

- Running example: Validate user id pipeline without exceptions
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.either;

import com.buildwithwaqar.patterns.functional.either.Railway;
/** Problem: validate an ID through parse→positive without try/catch ladders. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Either problem: railway ID validation ===");
    System.out.println(Railway.explain(Railway.label("42")));
    System.out.println(Railway.explain(Railway.label("nope")));
    System.out.println(Railway.explain(Railway.label("-1")));
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.either;

import dev.zazr.control.Either;

/** Railway-oriented programming with Zazr Either. */
public final class Railway {
    private Railway() {}

    public static Either<String, Integer> parseInt(String raw) {
        try {
            return Either.right(Integer.parseInt(raw));
        } catch (NumberFormatException e) {
            return Either.left("not an int: " + raw);
        }
    }

    public static Either<String, Integer> requirePositive(int n) {
        return n > 0 ? Either.right(n) : Either.left("not positive: " + n);
    }

    public static Either<String, String> label(String raw) {
        return parseInt(raw)
                .flatMap(Railway::requirePositive)
                .map(n -> "id-" + n);
    }
```

Solution (2)

```
    public static String explain(Either<String, String> result) {
        return switch (result) {
            case Either.Right(var v) -> "ok:" + v;
            case Either.Left(var err) -> "err:" + err;
        };
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.either;

import com.buildwithwaqar.patterns.functional.either.Railway;
/** Problem: validate an ID through parse→positive without try/catch ladders. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Either problem: railway ID validation ===");
    System.out.println(Railway.explain(Railway.label("42")));
    System.out.println(Railway.explain(Railway.label("nope")));
    System.out.println(Railway.explain(Railway.label("-1")));
  }
}
```

Walkthrough Output

```
=== Either problem: railway ID validation ===
ok:id-42
err:not an int: nope
err:not positive: -1
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Either/Railway.

Related Patterns

- Contrast Either/Railway with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Either/Railway → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.